ISO/IEC 27001 Evidence as Code

Continuous audit evidence from Terraform scans: technical report

Mir Mansoor Ahmed

https://github.com/Mirmansoorahmed/iso27001-evidence-as-code

October 2026





1 The problem

For a technical ISO 27001 control, the evidence is usually a screenshot: someone opens the Azure portal, finds one storage account, takes a picture and drops it into a folder named after the control. It shows one resource on one day, it does not say which commit or rule it came from, and nothing stops the file being swapped later.

If the infrastructure is Terraform and every change goes through a pull request, the configuration already has a history and a review. What is missing is the step from "the scan passed" to "here is evidence for A.8.24, from this commit, checked by this rule, and you can verify the file has not changed". That step is what I built.

2 What I built

An offline pipeline for a fictional Berlin payment institution, NordPay GmbH, whose Azure landing zone is written in Terraform. Everything is in one repository:

- **Scan.** On every pull request and every night, GitHub Actions runs Checkov (21 adopted rules) and Conftest with 8 Rego policies I wrote for NordPay's own rules (tags, EU regions, Owner role, backups, log retention).
- **Collect.** A Python collector turns every result, pass or fail, into a hashed evidence record.
- **Map.** One YAML catalogue maps the 29 rules to ISO/IEC 27001:2022 Annex A, BSI C5:2020, IT-Grundschutz modules and DORA articles.
- **Report.** A statement of applicability (SoA) for all 93 Annex A controls, a drift report, and an auditor pack that can be checked with `sha256sum` and `jq`.

Two extra steps exist but are off by default: opening a GitHub issue for each failing check (corrective action), and signing the pack with Sigstore Cosign. Neither was on for the committed run.

Commercial platforms such as Vanta and Drata watch the live environment and cover far more ground: people, vendors, policies. This project works earlier, on the code before it is merged, and its pack can be checked by someone with no account on anything.

3 Architecture

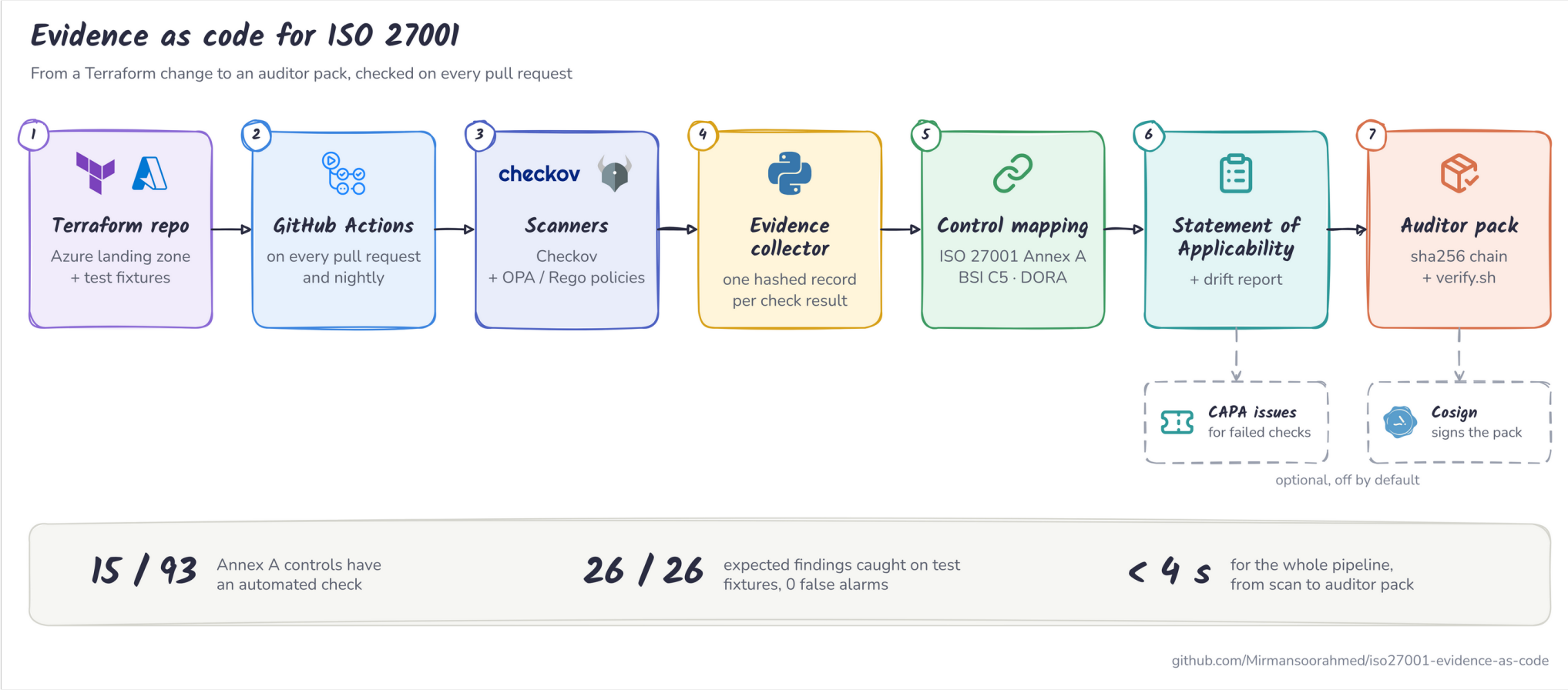


Figure 1. From a Terraform change to an auditor pack. The dashed boxes are opt-in and were off for the committed run. Product icons belong to their owners.

4 How it works

Evidence chain

Each record names the rule, the resource, the file, the commit, the result, its control ids and the SHA-256 of the raw scanner output it was built from, so an argument about what the scanner said can be settled by re-hashing. The committed run wrote 369 records. The pack lists every file with its hash in `MANIFEST.json` and carries the previous manifest, so each period's pack is chained to the last. `verify.sh` rechecks all of it with `sha256sum` and `jq` only. The demo flips one byte and confirms that verification fails. A hash chain proves the files did not change; it does not prove who made them, which is what the optional signature is for.

Control mapping

The mapping is my judgement, written down in one file so it can be reviewed like code. A TLS 1.2 check on a storage account supports A.8.24 (cryptography) and A.8.9 (configuration). The repository stores control ids only, never standard text. Denominators are honest: all 93 Annex A controls, all 121 C5 criteria, and for DORA a slice of 8 articles that I chose and name.

When every check for a control passes, the control is marked `partial`, never `automated-pass`. A backup check passing says nothing about whether a restore was ever tested, and I could not find one of the 15 where the scan was the whole control. A failing check does turn a control red straight away.

5 Results

All numbers are from the committed run (`eac demo`, 27 October 2026, 13:19:08–13:19:12 CET), read from `results/*.json` by the script that builds this PDF.

Measure	Result
Annex A controls with at least one automated check	15 / 93 (16.1%)
SoA on the clean landing zone: partial / manual / not applicable automated-pass is 0 by design (Section 4)	15 / 76 / 2
BSI C5:2020 criteria with a check	15 / 121 (12.4%)
DORA articles with a check, in my tracked slice	4 / 8
Own fixtures, TP / FP / FN / TN Self-check only: I wrote the fixtures and the policies together	26 / 0 / 0 / 270
TerraGoat (held out), adopted Checkov rules, TP / FP / FN / TN The miss: `CKV_AZURE_3` passes when the HTTPS setting is missing	17 / 0 / 1 / 3
Simulated drift (storage TLS 1.2 → 1.0), controls that lost evidence via `CKV_AZURE_44`	A.8.24 and A.8.9
Whole pipeline, including a 2.41 s scan	3.97 s

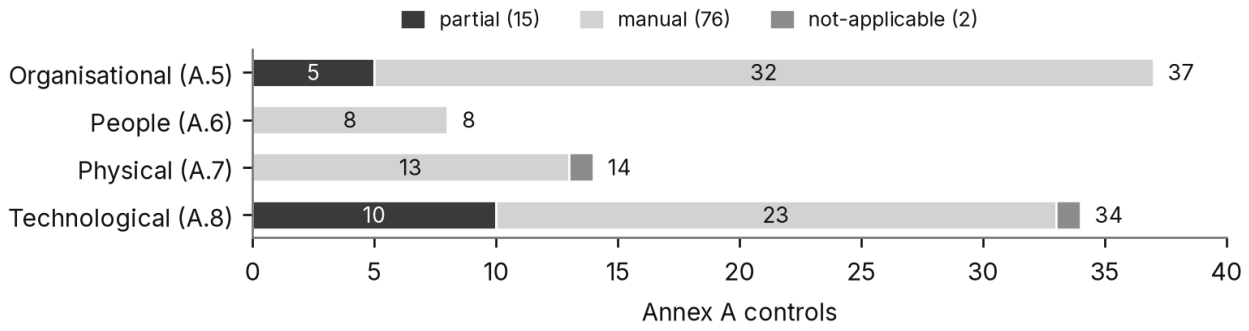


Figure 2. SoA status of all 93 Annex A controls by theme. People and physical controls have no automated check, as you would expect from a scan of code.

6 Limitations

- **Offline only.** It reads Terraform, not a live Azure subscription, so nothing here proves what is deployed. The drift is a fixture.
- **Azure only**, and no live scanner such as Prowler.
- **One independent test.** The perfect fixture score is circular; TerraGoat is the only outside check.
- **Absence passes.** Some Checkov rules pass when an attribute is missing, which is how TerraGoat's FN slipped through.
- **Unreviewed mapping.** No auditor has reviewed it, and no lawyer has checked the DORA/NIS2 framing.
- **Opt-in steps barely tested.** Corrective actions are tested only against a fake GitHub API, and signing not at all.

7 Next steps

- Scan `terraform show -json` plans, not only HCL, so variables and modules resolve.
- Add a read-only live mode (Prowler or Azure Policy) writing the same records, to compare intended and deployed state.
- Replace absence-as-pass checks with explicit Rego rules and score them on TerraGoat.
- Run corrective actions and signing end to end in a test repository.
- Have the mapping reviewed by an ISO 27001 / C5 auditor.

8 Kurzfassung (Deutsch)

Technische ISMS-Nachweise sind oft Screenshots: eine Momentaufnahme, schwer nachvollziehbar, später nicht prüfbar. Für das fiktive Zahlungsinstitut NordPay GmbH habe ich eine Offline-Pipeline gebaut, die Nachweise direkt aus dem Terraform-Code einer Azure-Landing-Zone erzeugt. Checkov und eigene Rego-Regeln prüfen den Code bei jedem Pull Request und nächtlich in GitHub Actions. Jedes Ergebnis wird ein gehashter Nachweis, zugeordnet zu ISO/IEC 27001 Annex A, BSI C5 und DORA. Das Auditor-Paket lässt sich allein mit `sha256sum` und `jq` prüfen.

15 von 93 Annex-A-Controls haben eine automatisierte Prüfung, bei C5 15 von 121. Keiner gilt allein durch einen Scan als erfüllt, weil jeder einen organisatorischen Rest hat. Gegen TerraGoat: TP 17, FP 0, FN 1. Grenzen: nur offline, nur Azure, Mapping nicht extern geprüft. Kein Zertifikat, keine Prüfungsaussage.

Sources.

1. ISO/IEC 27001:2022, Annex A. iso.org/standard/27001
2. BSI, Kriterienkatalog C5 – Einführung. bsi.bund.de (C5)
3. Regulation (EU) 2022/2554 (DORA). eur-lex.europa.eu/eli/reg/2022/2554/oj
4. Checkov (checkov.io); Conftest and OPA (conftest.dev, openpolicyagent.org)
5. TerraGoat, Bridgecrew, Apache-2.0, commit 729f8da62c6a. github.com/bridgecrewio/terragoat
6. Repository: https://github.com/Mirmansoorahmed/iso27001-evidence-as-code, PR #1, commit 183e10d